CS4052 Computación Paralela y Distribuida – 2026-II

Proyecto de Laboratorio – **Entrega final**

Caso 5: KNN en paralelo con MPI (mpi4py)

Clasificación de dígitos (*load_digits*)

Profesor: José Fiestas – Grupo 7

Integrante	**Participación**
Barzola Estrella, Johar Jared	100 %
Huaman Casma, Leonardo Fabian	100 %
Izaguirre Zavaleta, Luis Fernando	100 %

(100 % indica que el integrante contribuyó en forma equitativa al resto)

1. Introducción

KNN (*k-nearest neighbors*) es un algoritmo de clasificación supervisada. Para clasificar un punto nuevo, calcula su distancia a todos los puntos de entrenamiento, toma los k más cercanos y le asigna la etiqueta que más se repite entre ellos.

En este proyecto paralelizamos el código secuencial `knn_digits_sec.py` [2], que clasifica las imágenes del conjunto *load_digits* de scikit-learn [4]. El conjunto tiene $n = 1797$ imágenes de 8×8 píxeles, es decir $d = 64$ características, y 10 clases. El código separa los datos con `train_test_split` (20 % para prueba, `random_state=42`), lo que da $n_{tr} = 1437$ puntos de entrenamiento y $n_{te} = 360$ de prueba, y usa $k = 3$. Su precisión (*accuracy*) es 0,9833, que tomamos como referencia.

El trabajo más costoso es calcular las $n_{tr} \cdot n_{te}$ distancias, y cada una se puede calcular de forma independiente de las demás. Por eso es un problema naturalmente paralelizable [5].

Solución planteada. Usamos el paradigma de memoria distribuida (paso de mensajes), en Python con mpi4py [3]. El proceso 0 (maestro) tiene los datos al inicio. Siguiendo el DAG del enunciado [1] (Figura 1):

- se envían los datos de prueba a todos los procesos (`comm.bcast`);
- se reparten los datos de entrenamiento en bloques (`comm.scatter`);
- cada proceso calcula las distancias de todos los puntos de prueba a su bloque, las ordena y guarda sus k vecinos más cercanos por punto de prueba;
- el maestro recolecta esos candidatos (`comm.gather`), elige los k más cercanos de todos y aplica la votación por mayoría.

Si un punto es uno de los k vecinos más cercanos entre todos los datos de entrenamiento, también lo es dentro de su bloque: de lo contrario, ese bloque tendría al menos k puntos más cercanos, y el punto no podría ser vecino global. Por ello, al reunir los candidatos locales y seleccionar otra vez los k de menor distancia, se recuperan los mismos vecinos del algoritmo secuencial, salvo empates de distancia.

Para preparar este informe usamos un asistente de IA [10]; en la Bibliografía describimos para qué lo usamos y en qué partes influyó.

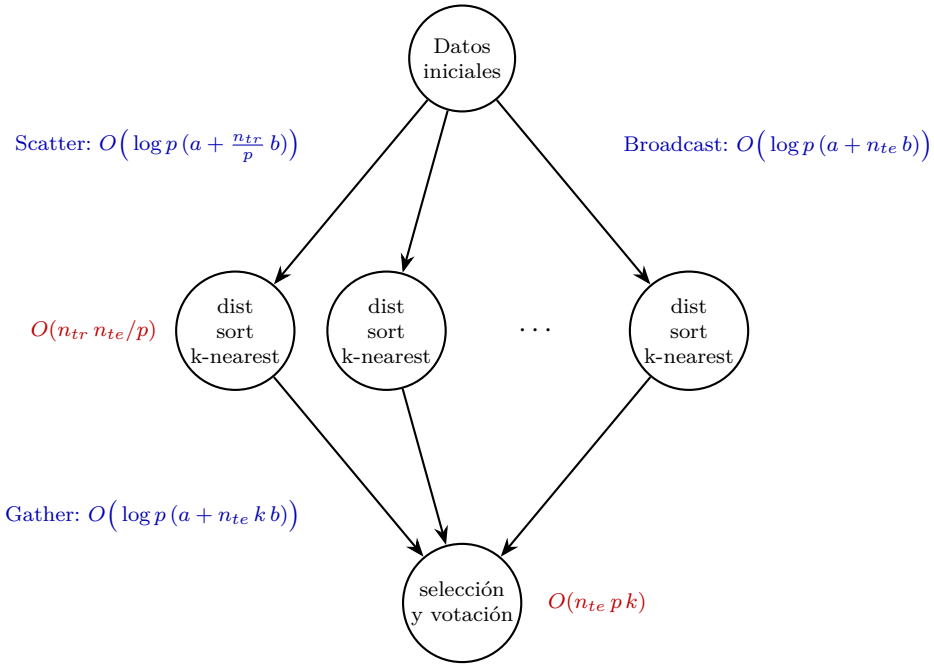


Figura 1: DAG del KNN en paralelo, adaptado de [1] con los costos de `scatter` y `gather` corregidos a $O(\log p)$ (Sección 2.5), confirmado por el profesor [5, 8]. En azul, comunicación; en rojo, cómputo. Aquí n_{te} cuenta puntos, con d constante.

2. Método

2.1. Algoritmo secuencial

Llamamos $n = n_{tr} + n_{te}$ al tamaño de los datos. La división 80/20 se mantiene fija, así que $n_{te} = c\,n$ y $n_{tr} = (1-c)\,n$ con $c \approx 0{,}2$. Con esto, el problema depende de un **solo tamaño** n, y $n_{tr}\,n_{te} = c(1-c)\,n^2 \approx 0{,}16\,n^2$. El tiempo secuencial se obtiene del código base [2] (Tabla 1).

Línea del código secuencial	**Operaciones**
`euclidean_distance(a, b)`	d restas, d productos, $d-1$ sumas y 1 raíz: $O(d)$
`distances = [... for x in X_train]`	$n_{tr} \cdot O(d)$
`np.argsort(distances)[:k]`	$O(n_{tr} \log n_{tr})$
`k_labels = [y_train[i] for i in k_indices]`	$O(k)$
`Counter(k_labels).most_common(1)`	$O(k)$
`y_pred = [knn_predict(x, ...) for x in X_test]`	lo anterior se repite n_{te} veces
`accuracy = np.mean(y_pred == y_test)`	$O(n_{te})$, una sola vez

Tabla 1: Operaciones de cada línea del código secuencial [2].

$$T_s(n) = O\big(n_{te}(n_{tr}\,d + n_{tr}\log n_{tr} + k) + n_{te}\big) = O\big(n^2(d + \log n)\big)$$

Como $d = 64 > \log_2 n \approx 10{,}8$, el cálculo de distancias domina al ordenamiento:

$$T_s(n) = O(d\,n^2)$$

2.2. Diseño con el método de Foster

La Figura 2 resume los cuatro pasos de Foster [5].

Particionamiento. Usamos descomposición del dominio sobre la data de mayor tamaño, la de entrenamiento ($n_{tr} > n_{te}$). La tarea primitiva es la distancia entre un punto de prueba y uno

de entrenamiento, lo que da $n_{tr} \cdot n_{te} = 517\,320$ tareas de costo $O(d)$ y de tamaño parecido, más de cuatro órdenes de magnitud por encima de $p = 16$ [5].

Comunicación. Cada tarea necesita sus dos puntos, y su distancia se usa luego para elegir los k vecinos de ese punto de prueba. Es decir, las tareas de una misma fila se combinan en una reducción. Contando solo las salidas, son del orden de $n_{tr}\, n_{te} \approx 5{,}2 \times 10^5$ canales hacia la selección. Esta comunicación es *overhead*: no existe en el secuencial.

Aglomeración. Agrupamos en una tarea todas las columnas de un bloque de puntos de entrenamiento. Así, cada proceso recibe solo su bloque y los datos de prueba, y envía k candidatos por punto de prueba en lugar de todas sus distancias. Contando un canal de Broadcast, uno de Scatter y uno de Gather por bloque, quedan del orden de $3p$ canales.

Mapeo. Asignamos un bloque por proceso. Como $n_{tr} = 1437$ no es múltiplo de p para $p = 2, 4, 8, 16$, los primeros n_{tr} mód p procesos reciben $\lceil n_{tr}/p \rceil$ puntos y el resto $\lfloor n_{tr}/p \rfloor$. Los bloques difieren en un punto como máximo, así que la carga queda balanceada y en el análisis usamos n_{tr}/p [5]. Con $p = 16$ son unos 48 canales.

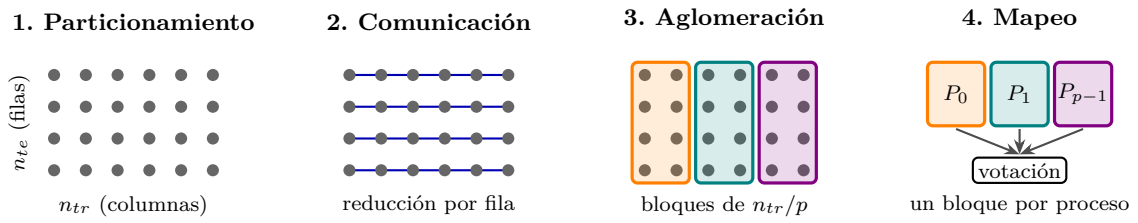


Figura 2: Método de Foster aplicado al KNN. Cada punto es una distancia entre un punto de prueba y uno de entrenamiento.

El enunciado también menciona repartir los datos de prueba. Esa alternativa (*scatter* de prueba y *broadcast* de entrenamiento) es válida y evita el paso 6 en el maestro. Sin embargo, seguimos la distribución del DAG, que reparte el conjunto de mayor tamaño.

2.3. PRAM

El Algoritmo 1 presenta el pseudocódigo con la notación PRAM del curso [5, 6]. No depende de p: los límites de los bucles se expresan solo con el tamaño del problema, y **pardo** va en el bucle sobre n_{tr}, que es la dimensión que se distribuye. El número de procesos aparece recién al ejecutar ese **pardo**, cuando cada proceso toma un bloque de valores de j y aplica la línea 6 solo a ellos; por eso el Gather reúne varias listas de candidatos por punto de prueba. Esto permite leer directamente el trabajo W y el *span* T_∞ (Sección 2.4).

Descripción de los pasos:

- **Paso 1:** el maestro envía los n_{te} datos de prueba y k a todos los procesos.
- **Paso 2:** el maestro reparte los datos de entrenamiento y sus etiquetas en bloques.
- **Paso 3:** se calculan las distancias de cada punto de prueba a los puntos de entrenamiento. Cada proceso calcula las de su bloque.
- **Paso 4:** por cada punto de prueba, cada proceso ordena las distancias de su bloque y guarda sus k vecinos locales (distancia y etiqueta).
- **Paso 5:** el maestro recolecta en un solo `Gather` los candidatos de todos los puntos de prueba, para no pagar la latencia n_{te} veces.
- **Paso 6:** por cada punto de prueba, el maestro elige los k pares de menor distancia entre todos los candidatos y aplica la votación. No se debe votar directamente con todos los candidatos: se votaría con más de k vecinos y el resultado podría diferir del secuencial.
- **Paso 7:** el maestro calcula la precisión comparando $\hat{y}$ con y_{te}.

Si lo vemos como un PRAM de memoria compartida, el algoritmo es CREW [5]: todos leen

Algoritmo 1: KNN distribuido con Broadcast/Scatter/Gather($X_{tr}, y_{tr}, X_{te}, y_{te}, k$)

Entrada: datos de entrenamiento X_{tr} ($n_{tr} \times d$) con etiquetas y_{tr}; datos de prueba X_{te} ($n_{te} \times d$) con etiquetas y_{te}; k vecinos

Salida: etiquetas predichas $\hat{y}[1, \ldots, n_{te}]$ y precisión en el proceso raíz

```
1  Broadcast(X_te, k, root = 0)                                   // paso 1
2  Scatter((X_tr, y_tr), root = 0)                                // paso 2
3  for i ← 1 a n_te do
4      for j ← 1 a n_tr pardo
5          D[j] ← sqrt(Σ_{l=1}^{d} (X_te[i,l] − X_tr[j,l])^2)     // paso 3
6      L[i] ← los k pares (D[j], y_tr[j]) de menor distancia      // paso 4
7  Gather(L, C, root = 0)                                         // paso 5
8  if es el proceso raíz then
9      for i ← 1 a n_te do
10         V ← los k pares de menor distancia en C[i]             // paso 6
11         ŷ[i] ← etiqueta más frecuente en V
12     accuracy ← (cantidad de i con ŷ[i] = y_te[i]) / n_te       // paso 7
13     return ŷ, accuracy
```

X_{te} al mismo tiempo y cada uno escribe solo en sus posiciones. En memoria distribuida, la lectura concurrente se reemplaza por el Broadcast del paso 1.

Optimización del paso 6

En el Algoritmo 1, el maestro hace solo todo el paso 6 y recibe todos los candidatos. Ambos costos crecen con p. Podemos evitarlo porque elegir los k menores es una operación asociativa: si $A \oplus B$ denota los k pares de menor distancia de $A \cup B$, entonces $(A \oplus B) \oplus C = A \oplus (B \oplus C)$. A diferencia de una suma [5], $\oplus$ no cuesta tiempo constante, sino $O(k)$ por punto de prueba; como k es constante, la cota no cambia. Así, la selección se puede hacer como una **reducción en árbol**, en la que cada nivel fusiona listas de k pares y solo reenvía k pares por punto de prueba. El Algoritmo 2 reemplaza las líneas 7 a 11 del Algoritmo 1; las líneas 12 y 13 no cambian. En mpi4py corresponde a `comm.reduce` con una operación propia, que para objetos de Python usa una reducción en árbol por defecto [3].

Algoritmo 2: Pasos 5 y 6 optimizados con una reducción en árbol

```
1  Reduce(L, C, op = ⊕, root = 0)                                 // pasos 5 y 6
2  if es el proceso raíz then
3      for i ← 1 a n_te do
4          ŷ[i] ← etiqueta más frecuente en C[i]
```

Conservamos ambas versiones: el Algoritmo 1 cumple con el `comm.gather` que pide el enunciado, y el Algoritmo 2 es la mejora que compararemos con él.

2.4. Trabajo y *span*

Al no depender de p, el pseudocódigo permite leer el trabajo W y el *span* T_∞, que es el tiempo con procesadores ilimitados, es decir, con un valor de j por procesador. El bucle sobre i es secuencial y el bucle sobre j es paralelo. Por cada i, la línea 5 cuesta $O(d)$ en el camino crítico.

Medida	Con `Gather` (Alg. 1)	Con reducción (Alg. 2)
Trabajo $W(n)$	$O(d\,n^2)$	$O(d\,n^2)$
Paso 3 en el camino crítico	$O(n_{te}\,d) = O(d\,n)$	$O(d\,n)$
Selección en el camino crítico	$O(n_{te}\,n_{tr}) = O(n^2)$	$O(n_{te}\,k \log n_{tr}) = O(k\,n \log n)$
Span $T_\infty(n)$	$O(n^2)$	$O\big(n(d + k \log n)\big)$
Paralelismo W/T_∞	$O(d)$	$O\big(d\,n/(d + k \log n)\big)$

Tabla 2: Trabajo y *span* de ambas versiones. Sin reducción, el maestro revisa todos los candidatos, por lo que su trabajo secuencial acota el paralelismo.

La Tabla 2 justifica la optimización: con `Gather`, el paso 6 del maestro queda en el camino crítico y limita el paralelismo a $O(d)$. Con la reducción, el paralelismo crece aproximadamente como $O(n)$.

Con el teorema de Brent, $T_p = O(W/p + T_\infty)$, pasamos del *span* a un tiempo con p procesos. Como Brent no incluye la comunicación, se suma aparte [5]:

$$T_p = O\Big(\frac{d\,n^2}{p} + n^2\Big) + T_{comm}, \qquad T'_p = O\Big(\frac{d\,n^2}{p} + n(d + k \log n)\Big) + T_{comm}$$

La primera cota es mayor que la de la Sección 2.5, y no es una contradicción. Brent acota la simulación del PRAM, donde el maestro revisa n_{tr} pares por punto de prueba. Al aglomerar en p bloques, cada bloque envía solo k candidatos, y el maestro revisa $p\,k$ pares. Por eso, en la Tabla 3, el paso 6 cuesta $O(n_{te}\,p\,k)$ y no $O(n_{te}\,n_{tr})$.

El bucle sobre i también es independiente [1]. Con **pardo** en i y en j, y con la reducción, el *span* bajaría a $O(d + k \log n)$. Dejamos **pardo** solo en j porque es la dimensión que se distribuye.

2.5. Métricas

Para la comunicación usamos el modelo del curso [5]:

$$T_{comm} = f(p)\,(\alpha + x\beta)$$

donde α es la latencia, β el tiempo de transferencia por unidad de dato, x la cantidad de datos y $f(p)$ depende de la topología. En el DAG, $a \equiv \alpha$ y $b \equiv \beta$.

Siguiendo la corrección del profesor [5, 8], tomamos $f(p) = \log p$ para las tres colectivas, es decir, **ninguna paga p veces la latencia ni p veces el término de datos**:

- **Broadcast:** cada proceso recibe el mensaje completo, de tamaño $x = n_{te}\,d$, con costo $\log p\,(a + n_{te}\,d\,b)$.
- **Scatter:** cada proceso recibe solo su bloque, de tamaño $x = \frac{n_{tr}}{p}\,d$, con costo $\log p\,(a + \frac{n_{tr}}{p}\,d\,b)$.
- **Gather:** el maestro recibe los k candidatos por punto de prueba de cada proceso, $x = n_{te}\,k$, con costo $\log p\,(a + n_{te}\,k\,b)$.

Un modelo más detallado describe estas colectivas como un árbol binomial de $\log_2 p$ niveles [7]: en el *scatter*, cada nivel reenvía la mitad de lo que tiene, y en el *gather* el maestro termina recibiendo físicamente $(p-1)$ bloques en vez de p. Esto puede hacer que la comunicación medida crezca algo más rápido que $\log p$, sobre todo en el *gather*; lo retomamos en la Sección 3.7 si las mediciones lo muestran.

La Tabla 3 muestra el costo de cada paso del Algoritmo 1.

Tiempo de ejecución. Sumando los pasos y usando $n_{te}, n_{tr} = O(n)$; los tres términos $a \log p$ de la Tabla 3 se absorben en uno solo, y el término de datos del *scatter*, $\frac{n}{p}\,d\,b \log p$, queda dominado

Paso	Líneas	Qué se ejecuta	Costo
1	1	Broadcast de los datos de prueba	$T_{comm_1} = O\big(\log p\,(a + n_{te}\,d\,b)\big)$
2	2	Scatter de los bloques	$T_{comm_2} = O\big(\log p\,(a + \frac{n_{tr}}{p}\,d\,b)\big)$
3 y 4	3–6	Distancias y ordenamiento	$T_{comp_1} = O\big(\frac{n_{te}\,n_{tr}}{p}(d + \log \frac{n_{tr}}{p})\big) = O\big(\frac{d\,n^2}{p}\big)$
5	7	Gather de k pares por punto	$T_{comm_3} = O\big(\log p\,(a + n_{te}\,k\,b)\big)$
6 y 7	8–13	Selección, votación y precisión	$T_{comp_2} = O(n_{te}\,p\,k)$

Tabla 3: Costo de cada paso del Algoritmo 1.

por el del *broadcast*, $n\,d\,b\log p$:

$$T_p(n,p) = O\Big(\frac{d\,n^2}{p} + k\,n\,p + a\log p + b\,n\log p\,(d+k)\Big)$$

A diferencia de la versión con el costo del *gather* sin corregir, aquí ningún término de comunicación crece como p: el único término lineal en p, $k\,n\,p$, viene del cómputo del maestro en el paso 6 (revisar $p\,k$ candidatos por punto de prueba), no de la comunicación.

Speedup. Con $S = T_s/T_p$, donde T_s es la misma rutina de distancias vectorizada, en un proceso y sin MPI. El curso pide especificar la versión secuencial considerada [5]. No usamos el código base [2] porque el *speedup* mezclaría la vectorización con el paralelismo; su tiempo se informa aparte.

$$S(n,p) = O\left(\frac{d\,n^2}{\dfrac{d\,n^2}{p} + k\,n\,p + a\log p + b\,n\log p\,(d+k)}\right)$$

Eficiencia. Con $E = S/p = T_s/(p\,T_p)$, y dividiendo entre $d\,n^2$:

$$E(n,p) = O\left(\frac{1}{1 + \dfrac{k\,p^2}{d\,n} + \dfrac{b\,(d+k)\,p\log p}{d\,n} + \dfrac{a\,p\log p}{d\,n^2}}\right) = O\left(\frac{1}{1 + \dfrac{p^2}{n}}\right)$$

En el último paso tomamos a, b, d y k como constantes y conservamos el término dominante, porque $p\log p \le p^2$. Este resultado no cambia respecto a cuando el *gather* todavía llevaba p en su término de datos: el término dominante, $k\,p^2/(d\,n)$, viene del cómputo del paso 6, no de la comunicación.

Con el paso 6 optimizado. La reducción reemplaza T_{comm_3} por $O\big(\log p\,(a + n_{te}\,k\,b)\big)$ y T_{comp_2} por $O(n_{te}\,k\log p)$, con lo que ya no queda ningún término que crezca linealmente con p:

$$T'_p(n,p) = O\Big(\frac{d\,n^2}{p} + k\,n\log p + a\log p + b\,n\,(d+k)\log p\Big), \qquad E'(n,p) = O\left(\frac{1}{1 + \dfrac{p\log p}{n}}\right)$$

Isoeficiencia. Usamos la función de *overhead* $T_0 = p\,T_p - T_s$: para mantener E constante, T_s debe crecer como T_0, y cada término de T_0 da una condición, de la que manda la más restrictiva [5]. Con a, b, d y k constantes, $T_0 = O(n\,p^2 + n\,p\log p + p\log p)$. Comparando cada término con $T_s = O(n^2)$ se obtienen $n \propto p^2$, $n \propto p\log p$ y $n \propto \sqrt{p\log p}$, así que n debe crecer al menos como p^2: $n = \Omega(p^2)$. Con la reducción, $T'_0 = O(n\,p\log p + p\log p)$ y basta $n = \Omega(p\log p)$, de modo que esa versión es más escalable. Ambas condiciones se validan con las mediciones en la Sección 3.7.

Interpretación. Las expresiones $O(\cdot)$ omiten las constantes de tiempo de cada operación. El curso toma a y b del orden de 10^{-7}–10^{-9} s como referencia [5]; los valores de nuestra máquina se miden en la Sección 3.1. Para p pequeño esperamos que el tiempo esté dominado por las distancias

y que el algoritmo sea *compute bound*. Al aumentar p, ese trabajo disminuye como $1/p$, pero los costos del maestro y de la comunicación no, y la eficiencia tiende a bajar.

FLOPs. Por la fórmula de la distancia euclidiana, cada distancia hace d restas, d productos, $d-1$ sumas y 1 raíz, es decir $3d$ operaciones. La raíz cuenta como una operación aunque sea más lenta, porque los FLOP contabilizan igual operaciones distintas [5]. En la región paralelizable:

$$F(n) = 3\,d\,n_{tr}\,n_{te}, \qquad \text{FLOPs}(p) = \frac{F(n)}{T_p}$$

También mostramos $F(n)/\max_q t_{comp}^{(q)}$, que excluye la comunicación. Para $n = 1797$ se tiene $F = 3 \cdot 64 \cdot 517\,320 \approx 9{,}93 \times 10^7$ operaciones.

Normalización. Para comparar la teoría con las mediciones, reemplazamos cada término de T_p por un coeficiente en segundos que absorbe d, k, c, a, b y el costo real de cada operación:

$$\hat{T}(n,p) = c_1\frac{n^2}{p} + c_2\,n\,p + c_3\,n\log_2 p + c_4\log_2 p$$

Los coeficientes $c_i \geq 0$ se ajustan por mínimos cuadrados con las mediciones. Para la versión con reducción, $\hat{T}'$ no lleva el término c_2. Cada coeficiente tiene un significado físico: c_1 refleja el tiempo por operación de distancia, c_2 el costo de los candidatos en el maestro, c_3 el tiempo por dato y c_4 la latencia. Por eso comparamos c_3 y c_4 con los b y a medidos con un ping-pong [5]. Además ajustamos por separado T_{comp} y T_{comm}, que se miden por separado.

Número óptimo de procesos. Derivando respecto de p e igualando a cero (en $\hat{T}$ despreciamos además los términos logarítmicos, menores que $c_2\,n\,p$):

$$p^* \approx \sqrt{\frac{c_1\,n}{c_2}} \qquad \text{y, con reducción,} \qquad p^{*\prime} \approx \frac{\ln 2\;c_1\,n^2}{c_3\,n + c_4}$$

En ambos casos son mínimos, porque la segunda derivada es positiva en ese punto: $\hat{T}''(p^*) = 2\,c_1\,n^2/p^{*3} > 0$, y $(\hat{T}')''(p^{*\prime}) = (c_3\,n + c_4)/(\ln 2\;p^{*\prime 2}) > 0$. Los compararemos con el punto en que se cruzan las curvas de cómputo y comunicación [5].

3. Resultados y análisis

Todos los valores de esta sección son la mediana de 5 repeticiones, medidas en una sola máquina. Las figuras y las cifras se generan con los scripts del repositorio a partir de los archivos CSV de `results/final`.

3.1. Entorno experimental

El clúster Khipu no estuvo disponible por un problema de *firewall* de la universidad, y el profesor autorizó medir en una laptop [8]. La Tabla 4 resume el entorno.

Datos. Con $n = 1797$ se usan los datos originales. Para $n < 1797$ tomamos submuestras estratificadas con semilla fija. Para $n > 1797$ replicamos las imágenes con un ruido entero pequeño, recortado al rango 0–16 de los píxeles. Estos datos ampliados solo sirven para medir tiempos, porque el costo de una distancia no depende de los valores. Su precisión (1,0) no se reporta: el conjunto de prueba contiene copias casi idénticas de imágenes de entrenamiento.

Cómo se mide. No usamos barreras. Cada proceso toma `MPI.Wtime()` antes y después de cada fase, y los tiempos se reúnen después de la parte medida. T_p lo mide el maestro, desde antes del Broadcast hasta después de la votación. T_{comm} también sale del maestro, y T_{comp} es el máximo entre procesos. Como no hay barreras, T_{comm} incluye la espera por los procesos más lentos. Cada

Procesador	Intel Core i7-13620H: 10 núcleos físicos de dos tipos (6 de rendimiento con 2 hilos cada uno y 4 de eficiencia), 16 hilos
Memoria y sistema	16 GB de RAM, Windows 11
Software	Python 3.14.6, NumPy 2.5.4 [9], mpi4py 4.1.2 [3], Intel MPI 2021.19 (paquete `impi-rt`), scikit-learn 1.9.1 [4]
Malla	$p \in \{1, 2, 4, 8, 16\}$ y $n \in \{450, 900, 1350, 1797, 3600, 7200\}$, con las dos versiones: 300 corridas
Repeticiones	5 por configuración, cada una en un `mpiexec` distinto; se reporta la mediana

Tabla 4: Entorno de las mediciones.

corrida ejecuta el algoritmo una vez sin medir y mide la segunda ejecución (Sección 3.2). Fijamos un hilo por proceso en NumPy, para que la implementación no se vuelva híbrida.

Memoria compartida dentro de un nodo. Todos los procesos corren en una misma máquina, así que MPI transmite los mensajes por memoria compartida y no por red. El modelo sigue siendo de memoria distribuida, porque cada proceso tiene su propia memoria y solo se comunica con mensajes, pero a y b salen menores que entre nodos.

Latencia y tiempo por dato. Con un ping-pong entre 2 procesos, con `comm.send` y `comm.recv` y mensajes de 1 a 2^{20} datos `float64`, ajustamos $t(m) = a + b\,m$ [5]:

$$a = 3{,}6 \times 10^{-5}\ \text{s}, \qquad b = 5{,}8 \times 10^{-9}\ \text{s por dato } (\approx 1{,}4\ \text{GB/s})$$

La latencia es mucho mayor que la referencia de 10^{-7}–10^{-9} s del curso, porque cada mensaje paga la serialización de objetos de Python que hacen las funciones en minúscula de mpi4py [3]. Entre 16 384 y 65 536 datos el tiempo salta de 0,09 a 1,3 ms, probablemente porque MPI cambia de protocolo para los mensajes grandes.

Limitación del hardware. Con $p = 8$ ya hay procesos en núcleos de eficiencia, que son más lentos, y con $p = 16$ hay más procesos que núcleos físicos. No fijamos los procesos a núcleos, así que Windows decide dónde corre cada uno. Como T_{comp} es el máximo entre procesos, el más lento marca el tiempo. Este efecto aparece en todos los resultados con $p \geq 8$.

3.2. Desarrollo en versiones beta

El código se desarrolló en tres versiones (Tabla 5). Cada una se validó contra el código base [2]: para $n = 1797$ y $p = 1, 2, 3, 4, 7, 8, 16$, las tres dieron exactamente las mismas 360 predicciones, con precisión 0,9833. Los valores $p = 3$ y $p = 7$ se incluyeron para comprobar bloques de distinto tamaño. La versión secuencial vectorizada que se usa como T_s también dio las mismas predicciones. Los empates de distancia se resuelven por índice global, para que el resultado no dependa del reparto.

Versión	**Contenido**
Beta 1	`comm.bcast`, `comm.scatter` y `comm.gather` con bloques desiguales (Algoritmo 1)
Beta 2	Beta 1 con medición separada de cómputo y comunicación, y conteo de FLOP
Beta 3	Paso 6 optimizado con `comm.reduce` y una operación propia (Algoritmo 2)

Tabla 5: Versiones beta del código.

Durante el desarrollo aparecieron tres problemas que afectaban las mediciones o el costo teórico:

- **Speedup superlineal falso.** La primera medición daba $E > 1$. La causa era que `(X_b - x) ** 2` crea arreglos temporales en cada iteración [9]: en bloques mayores a unos 0,7 MB, cada distancia costaba unas 4 veces más, lo que penalizaba a los valores pequeños de p. Se corrigió reutilizando un mismo arreglo.

- **Espera de arranque.** No usamos barreras para medir, porque agregan *overhead*. Sin ellas, el maestro empezaba a contar mientras otros procesos seguían importando librerías, y esa espera aparecía como comunicación. Se corrigió ejecutando el algoritmo una vez sin medir y midiendo la segunda ejecución.
- **Costo del paso 6.** La selección se implementó ordenando los $p\,k$ candidatos, para desempatar por índice. Esto agrega un factor $\log(p\,k)$ a T_{comp_2} en la Tabla 3, que en este rango de p es pequeño.

Antes de las mediciones finales se repitió la validación de las tres versiones, con el mismo resultado. Durante la primera corrida completa, una ejecución de T_s se quedó colgada después de guardar su resultado, probablemente porque la laptop entró en suspensión. Descartamos esa medición, que daba un tiempo tres veces menor que las demás, y agregamos al script de experimentos un reintento para ese caso. La corrida se completó después sin problemas.

3.3. Tiempos de ejecución, cómputo y comunicación

La Figura 3 muestra los tres tiempos frente a p, en escala logarítmica.

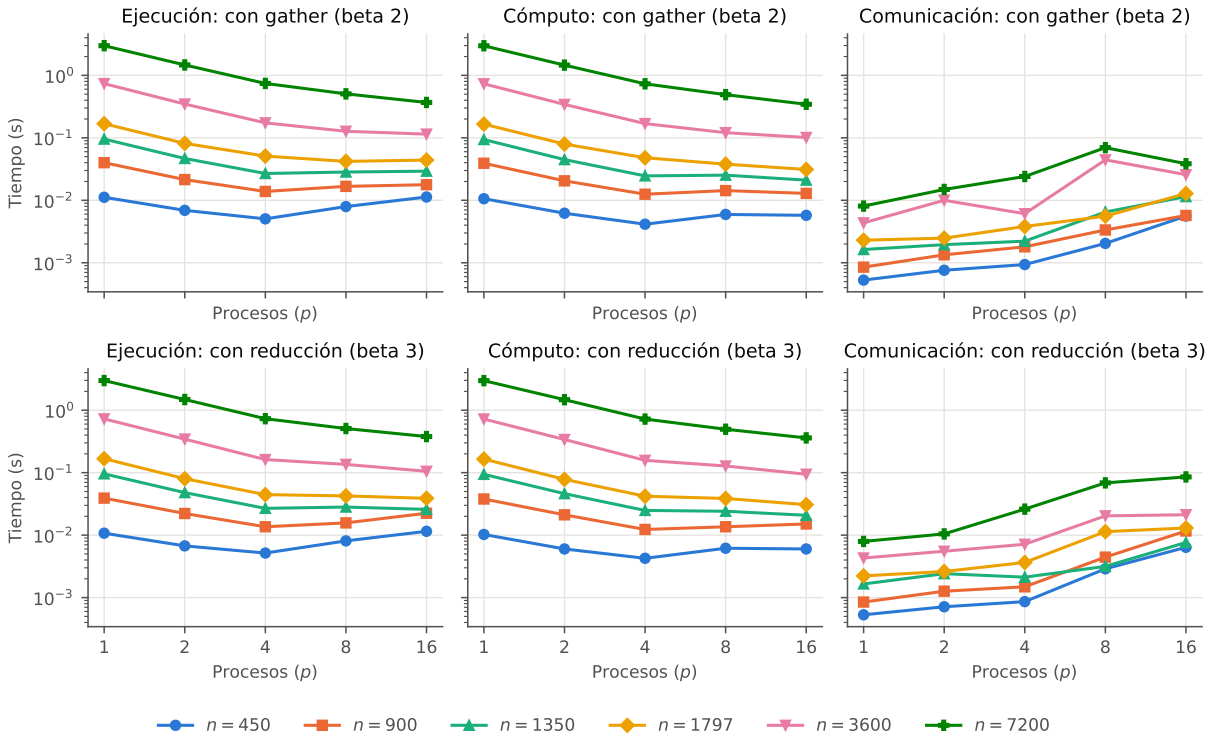


Figura 3: Tiempo de ejecución T_p, de cómputo ($\max_q t_{comp}^{(q)}$) y de comunicación (en el maestro) frente a p, para cada n. Arriba, con `Gather`; abajo, con la reducción.

Ejecución. Con $n \geq 3600$, T_p baja hasta $p = 16$: con $n = 7200$, de 2,98 s a 0,37 s. Con $n \leq 900$, el mínimo está en $p = 4$ y después el tiempo sube: con $n = 450$, pasa de 5,0 ms en $p = 4$ a 11,3 ms en $p = 16$. Con poco trabajo por proceso, el costo de agregar procesos supera la ganancia. Con $n = 1350$ y $n = 1797$ la curva queda casi plana desde $p = 4$: con $n = 1797$ y `Gather`, 51, 42 y 44 ms en $p = 4$, 8 y 16.

Cómputo. Hasta $p = 4$ se reduce casi a la mitad cada vez que se duplica p. Por ejemplo, con $n = 7200$ pasa de 2,98 a 1,46 y a 0,73 s. De $p = 4$ a $p = 8$ solo baja 1,5 veces (0,49 s), y de $p = 8$ a $p = 16$, 1,4 veces (0,34 s). Es la limitación del hardware de la Sección 3.1: desde $p = 8$ hay procesos en núcleos más lentos. En nuestras mediciones, este es el principal límite del *speedup*, más que la comunicación.

El cómputo incluye el ordenamiento del paso 4, que resultó caro: con $p = 1$ ocupa alrededor del 40 % del tiempo de las distancias más el ordenamiento (con $n = 1797$, 70 ms de 164 ms). La Sección 2.1 supuso que el ordenamiento era menor porque $d > \log n$, y eso es cierto en cantidad de operaciones. Pero cada operación del ordenamiento cuesta mucho más que una suma o un producto vectorizados.

Comunicación. Crece con p para todo n: con $n = 1797$ y `Gather`, de 2,5 ms en $p = 2$ a 12,8 ms en $p = 16$. Con $p = 1$ no es cero (2,3 ms con $n = 1797$), porque las colectivas en minúscula serializan los datos aunque haya un solo proceso [3]; el Scatter solo ya toma 1,7 ms. Algunos puntos son irregulares (con $n = 3600$ y $p = 8$, 44 ms), porque la comunicación incluye la espera por los procesos más lentos.

Granularidad. La Figura 4 muestra $G = T_{comp}/T_{comm}$ [5]. Baja al aumentar p y sube al aumentar n, como predice el modelo: el cómputo crece como n^2 y los datos enviados como n. G es mayor que 1 en todas las configuraciones salvo $n = 450$ con $p = 16$, donde llega a 1 con `Gather` y baja apenas de 1 con la reducción. Es justamente donde el tiempo total ya subía.

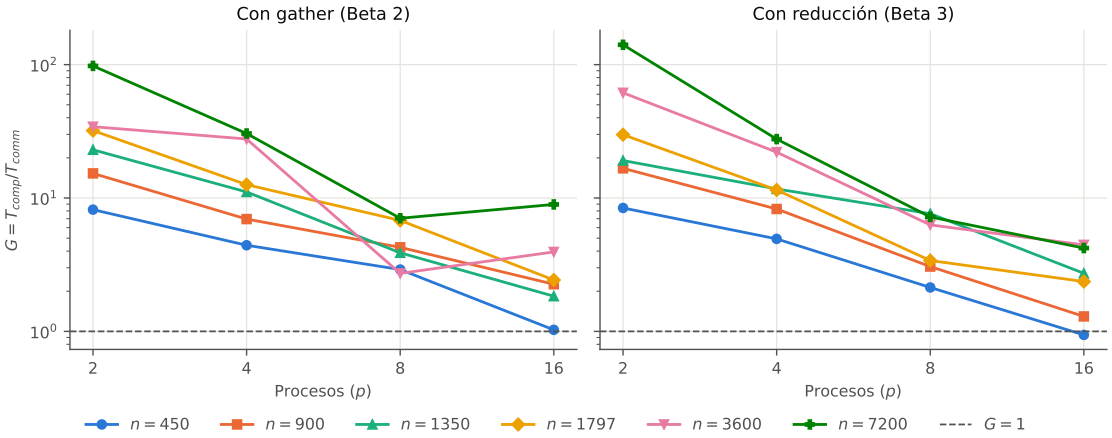


Figura 4: Granularidad $G = T_{comp}/T_{comm}$. Con $G < 1$ dominaría la comunicación.

Precisión. Con los datos originales y sus submuestras, la precisión es la misma para todo p y para las dos versiones (Figura 5): repartir el trabajo no cambia el resultado. Las diferencias entre valores de n vienen de los conjuntos de prueba, que tienen 90, 180, 270 y 360 puntos, no del paralelismo.

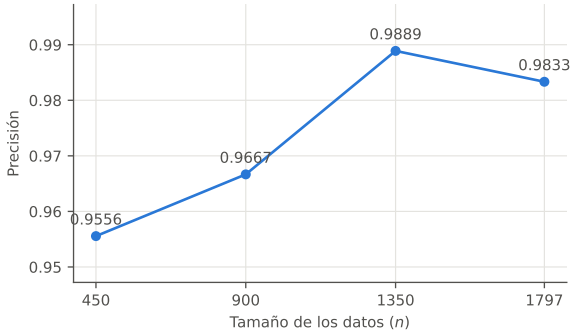


Figura 5: Precisión frente a n con los datos originales. Es la misma para todo p y en las dos versiones.

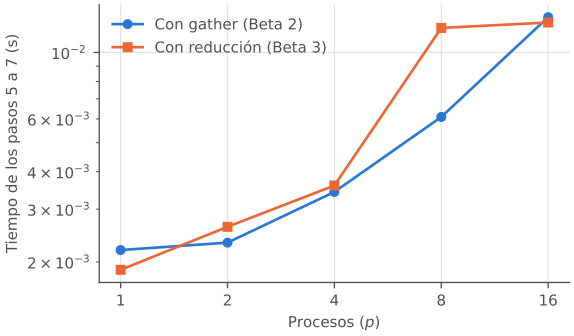


Figura 6: Tiempo de los pasos 5 a 7 (recolección, selección y votación) con $n = 1797$.

3.4. Comparación con la complejidad teórica

Cómo se normalizó. Para cada versión, ajustamos los coeficientes $c_i \geq 0$ de $\hat{T}$ (Sección 2.5) con las 30 medianas de T_p, por mínimos cuadrados no negativos. Minimizamos el error relativo, para que los tiempos de milisegundos pesen lo mismo que los de segundos. Cada c_i está en segundos por unidad de su término, es decir, la expresión teórica se multiplica término a término por un costo medido. La Tabla 6 muestra los coeficientes y la Figura 7 superpone $\hat{T}$ a las mediciones.

Versión	c_1 (s)	c_2 (s)	c_3 (s)	c_4 (s)	**Error medio**
`Gather`	$5{,}18 \times 10^{-8}$	$1{,}12 \times 10^{-6}$	$1{,}02 \times 10^{-7}$	$4{,}01 \times 10^{-4}$	8,1 %
Reducción	$4{,}87 \times 10^{-8}$	—	$3{,}95 \times 10^{-6}$	$1{,}95 \times 10^{-4}$	11,5 %

Tabla 6: Coeficientes de $\hat{T}(n,p) = c_1\, n^2/p + c_2\, n\, p + c_3\, n \log_2 p + c_4 \log_2 p$ y error relativo medio del ajuste.

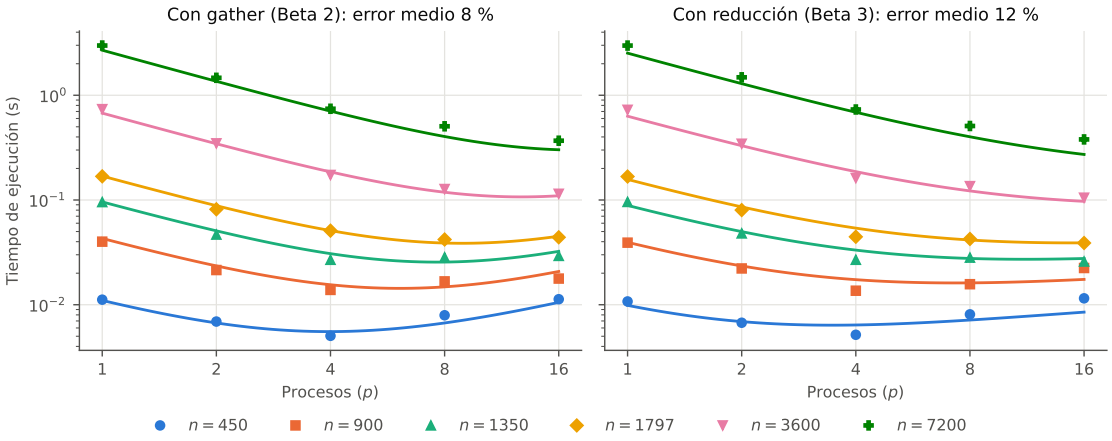


Figura 7: Tiempo de ejecución medido (puntos) y modelo normalizado $\hat{T}$ (líneas).

Significado de cada coeficiente.

- c_1: como $n_{tr}\, n_{te} \approx 0{,}16\, n^2$, equivale a $3{,}2 \times 10^{-7}$ s por distancia, incluida su parte del ordenamiento. Dividido entre los $3d = 192$ FLOP de una distancia, son $1{,}7 \times 10^{-9}$ s por FLOP, es decir 0,59 GFLOP/s. Es lo que se mide con $p = 1$: entre 0,53 y 0,64 GFLOP/s (Figura 10).
- c_3: el término de datos dominante es el del Broadcast, con $n_{te}\, d = 12{,}8\, n$ datos. Con el b del ping-pong, el modelo predice $12{,}8\, b = 7{,}5 \times 10^{-8}$ s. El ajuste da $1{,}0 \times 10^{-7}$ s, del mismo orden.
- c_4: con tres colectivas que pagan $a \log p$ cada una, el modelo predice $3a = 1{,}1 \times 10^{-4}$ s. El ajuste da 3,7 veces más, porque c_4 absorbe todo lo que crece como $\log p$ y no está modelado: la serialización y la espera por los procesos lentos. En cambio, al ajustar solo T_{comm}, el coeficiente de latencia es $3{,}5 \times 10^{-5}$ s, prácticamente el a del ping-pong ($3{,}6 \times 10^{-5}$ s).
- c_2: debería ser el costo de la selección en el maestro. Pero la selección medida con $n = 1797$ y $p = 16$ toma 4,7 ms, lo que da unos $1{,}6 \times 10^{-7}$ s por unidad de $n\, p$: el ajuste da 7 veces más. c_2 absorbe todo lo que crece linealmente con p, sobre todo el cómputo que deja de escalar desde $p = 8$.
- En la reducción, c_3 es 40 veces mayor que con `Gather`. En $\hat{T}'$, el término de las fusiones $\oplus$, $k\, n \log p$, y el de datos comparten la misma columna $n \log p$. Así que c_3 mide sobre todo las fusiones, que se hacen en Python por cada punto de prueba.

Error. El modelo reproduce la forma de las curvas, con su mínimo, y tiene un error medio de 8 % con `Gather` y 12 % con la reducción. Las mayores desviaciones están en $p = 8$ con n grande:

con $n = 7200$, el modelo da 0,40 s y se midió 0,51 s. Es el punto en que entran los núcleos de eficiencia.

Cómputo y comunicación por separado. La Figura 8 ajusta cada fase con sus propios términos. El cómputo tiene un error de 10 a 11 %. La comunicación tiene un error de 28 a 35 %: son tiempos de milisegundos, con ruido, y que incluyen esperas. En el ajuste de la comunicación con `Gather` agregamos el término $n(p-1)$ del modelo de árbol binomial [7], y su coeficiente no es nulo ($1{,}8 \times 10^{-7}$ s). Es decir, la comunicación medida crece más rápido que $\log p$, como anticipamos en la Sección 2.5: el maestro recibe físicamente $p-1$ bloques. La forma $\log p$ del curso describe bien la latencia, pero es una cota inferior para el término de datos del `gather`.

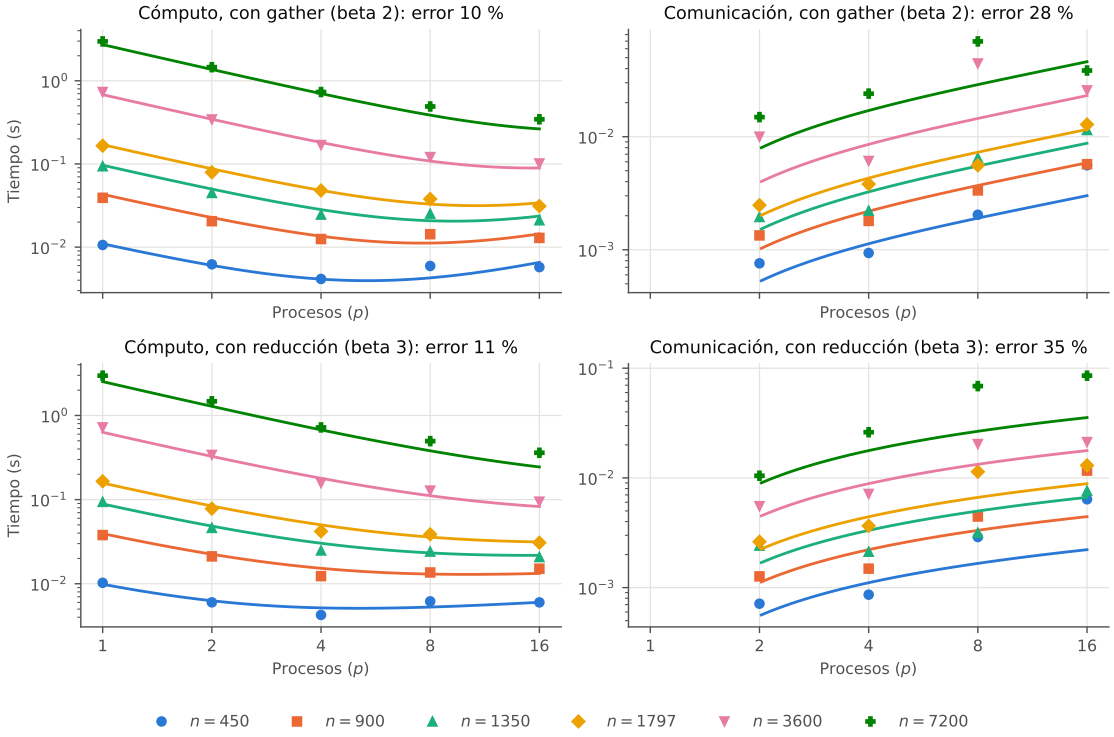


Figura 8: Cómputo y comunicación medidos (puntos) y su ajuste por separado (líneas).

3.5. Número óptimo de procesos

La Tabla 7 compara p^*, calculado con los coeficientes de la Tabla 6, con el p de menor tiempo medido.

	Con `Gather`		**Con reducción**	
n	p^* estimado	mejor p medido	$p^{*\prime}$ estimado	mejor p medido
450	4,6	4	3,5	4
900	6,4	4	7,3	4
1350	7,9	4	11,1	16
1797	9,1	8	15,0	16
3600	12,9	16	30,4	16
7200	18,2	16	61,2	16

Tabla 7: Número óptimo de procesos estimado con el modelo y mejor p de la malla medida.

Con `Gather`, $p^* = \sqrt{c_1 n / c_2}$ crece como $\sqrt{n}$. Coincide con el mejor p medido, dentro de la resolución de la malla (potencias de 2), para $n = 450$ y para $n \geq 1797$. Para $n = 900$ y $n = 1350$

lo sobrestima, pero allí el mínimo es plano: con $n = 1350$, $p = 4$ y $p = 8$ difieren en un 6 % (26,8 frente a 28,3 ms). Con la reducción, $p^{*\prime}$ crece casi como n. Para $n \geq 1797$ queda en el borde de la malla o fuera de ella, lo que coincide con que el mejor p medido sea 16, pero no se puede verificar sin más núcleos. En ambos casos $\hat{T}''(p^*) > 0$, así que son mínimos.

Para el conjunto original ($n = 1797$), el número óptimo de procesos es $p = 8$: el modelo da 9,1 y la medición, 8.

Las curvas de cómputo y comunicación no se cruzan en este rango (Figura 3): la comunicación medida queda por debajo del cómputo, salvo con $n = 450$ y $p = 16$, donde se igualan. Así que, aquí, el criterio del cruce [5] solo marca el límite para el n más pequeño. Para el resto, el óptimo lo fijan el costo que crece con p en el maestro y la limitación del hardware.

3.6. *Speedup* y FLOPs

T_s y código base. Para $n \geq 900$, T_s difiere en menos de 3 % de la corrida con $p = 1$ (con $n = 1797$, 0,166 frente a 0,168 s), así que el costo de MPI con un proceso es pequeño. Con $n = 450$ la diferencia es de 9 %, porque el tiempo total es de solo 10 ms. El código base [2] tarda 3,42 s con $n = 1797$, 20,6 veces más que T_s: eso es lo que aporta la vectorización. Juntando ambas cosas, la mejor versión paralela es entre 80 y 90 veces más rápida que el código base, pero de ese factor el paralelismo aporta solo unas 4 veces. Por eso el *speedup* se mide contra T_s y no contra el código base.

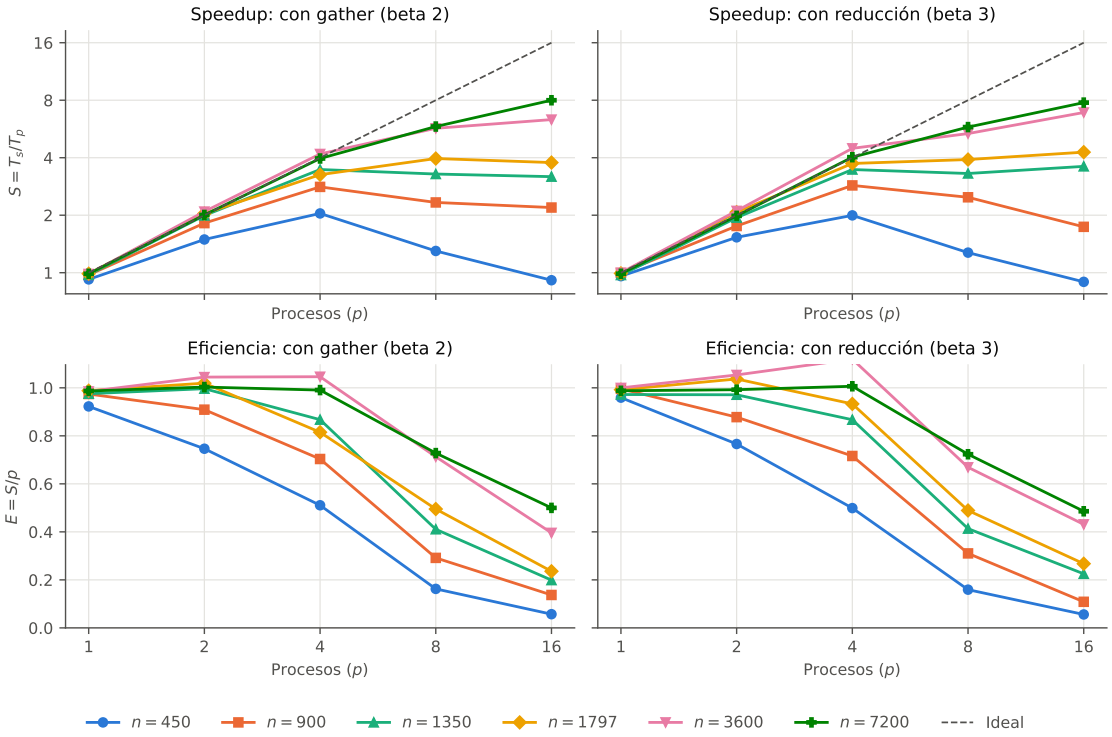


Figura 9: *Speedup* $S = T_s/T_p$ (arriba, escala logarítmica) y eficiencia $E = S/p$ (abajo). La línea punteada es el *speedup* ideal.

Speedup. La Figura 9 muestra que es casi lineal hasta $p = 4$ para $n \geq 1350$. El máximo es 8,00 con `Gather` y 7,77 con la reducción, ambos con $n = 7200$ y $p = 16$. Con los datos originales ($n = 1797$) llega a 3,96 en $p = 8$ con `Gather` y a 4,28 en $p = 16$ con la reducción. Con $n = 3600$ y $p = 2$ a 4, la eficiencia supera ligeramente 1 (hasta 1,05 con `Gather` y 1,12 con la reducción). Probablemente es un efecto de caché, porque los bloques más pequeños caben mejor en ella; no lo verificamos. No es el error de la Sección 3.2, que daba un factor de 4 y ya está corregido.

FLOPs. La Figura 10 muestra F/T_p y $F/\max_q t_{comp}^{(q)}$. Como F no depende de p, F/T_p sigue la forma del *speedup*: de 0,53 a 0,64 GFLOP/s con $p = 1$ hasta 4,33 GFLOP/s con `Gather` y 4,20 con la reducción ($n = 7200$, $p = 16$). Con $n = 1797$ el máximo es 2,37 GFLOP/s, en $p = 8$. Si se excluye la comunicación, con $n = 7200$ y $p = 16$ se llega a 4,9 GFLOP/s. Estos valores subestiman el trabajo real, porque F solo cuenta las distancias y no el ordenamiento.

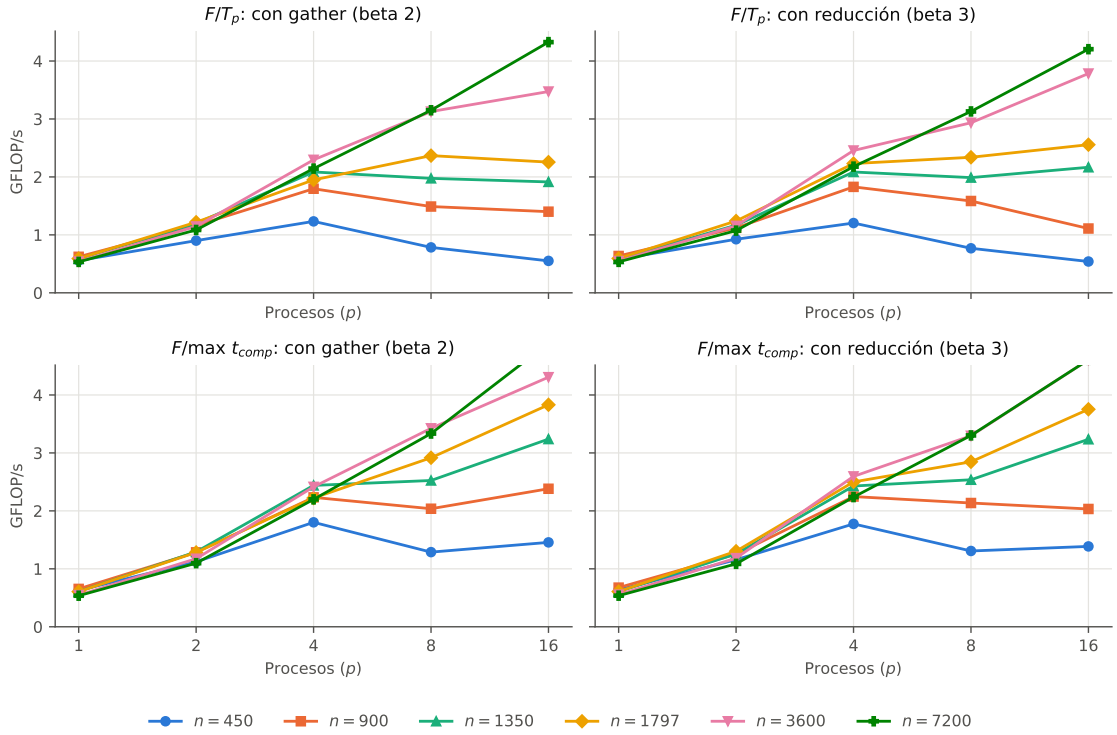


Figura 10: FLOP por segundo en la región paralelizable, con el tiempo total (arriba) y solo con el de cómputo (abajo).

3.7. Escalabilidad

Eficiencia con n variable. En la Figura 9 (abajo), la eficiencia baja al aumentar p y sube al aumentar n. Con $p = 16$ va de 0,06 con $n = 450$ a 0,50 con $n = 7200$. El algoritmo solo mantiene su eficiencia si el problema crece junto con p, como predice $E = O(1/(1 + p^2/n))$.

Validación de la isoeficiencia. Seguimos el método del curso [5]: escalar n según la condición y comprobar si E se mantiene. Medimos dos series de pares (n, p), con $n/p^2 \approx 28$ y con $n/(p \log_2 p) \approx 56$. Cada serie sirve además de control para la otra versión. La Tabla 8 y la Figura 11 comparan la E medida con la que predice el modelo normalizado, $\hat{E} = \hat{T}(n, 1)/(p\,\hat{T}(n, p))$.

		Con `Gather`		**Con reducción**	
Condición	(n, p)	E medida	$\hat{E}$	E medida	$\hat{E}$
$n \propto p^2$	(450, 4)	0,511	0,496	0,499	0,385
	(1797, 8)	0,495	0,545	0,489	0,474
	(7200, 16)	0,500	0,558	0,486	0,580
$n \propto p \log_2 p$	(450, 4)	0,511	0,496	0,499	0,385
	(1350, 8)	0,411	0,469	0,413	0,401
	(3600, 16)	0,396	0,385	0,431	0,407

Tabla 8: Eficiencia medida y del modelo normalizado en las series de isoeficiencia.

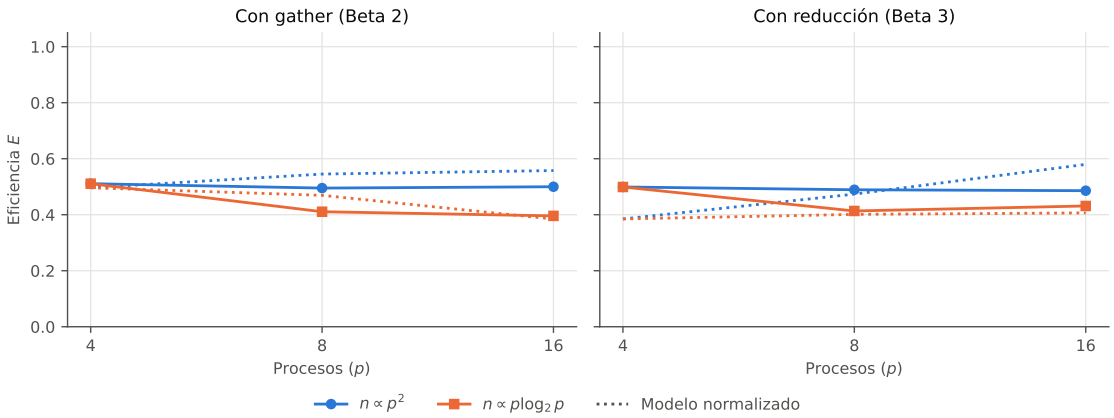


Figura 11: Eficiencia en las series de isoeficiencia. Línea continua: medida; punteada: modelo normalizado.

- **`Gather`, condición $n \propto p^2$: se cumple.** E se mantiene en 0,51, 0,50 y 0,50 mientras p pasa de 4 a 16, con una variación de unos 3 %. Esto valida con los datos la condición $n = \Omega(p^2)$ de la Sección 2.5.
- **`Gather`, control con $n \propto p \log p$: E cae** de 0,51 a 0,41 y a 0,40. Hacer crecer n más lento que p^2 no alcanza para esta versión, como predice el modelo.
- **Reducción, condición $n \propto p \log p$: no se cumple del todo.** E baja de 0,50 a 0,41 y sube a 0,43, una caída de 14 a 17 %. Con $n \propto p^2$ se mantiene (0,50, 0,49 y 0,49). La reducción elimina el término $k\,n\,p$ de la selección en el maestro, pero ese término medido es pequeño (la selección toma menos de 5 ms con $n = 1797$ y $p = 16$). La pérdida de eficiencia viene, sobre todo, del cómputo que deja de escalar desde $p = 8$ (Sección 3.3). Ese efecto afecta igual a las dos versiones y crece más rápido que $p \log p$. Por eso, en este rango, la reducción no muestra la mejor escalabilidad que predice la teoría.

El modelo normalizado sigue las series medidas con una diferencia de hasta 0,06 con `Gather` y de hasta 0,11 con la reducción. Dos advertencias: las condiciones de isoeficiencia son asintóticas, y cada serie tiene solo tres puntos, hasta $p = 16$. Además, con $p = 16$ hay más procesos que núcleos físicos. Para separar el algoritmo del hardware habría que repetir estas series en una máquina con al menos 16 núcleos iguales, como Khipu o las PC del laboratorio L507.

3.8. Mejoras a la solución

`Gather` frente a la reducción. La mejora que implementamos (Algoritmo 2) no da una ganancia clara en las mediciones. Los tiempos totales de las dos versiones difieren poco y en ambas direcciones. Con $n = 1797$ y $p = 16$ gana la reducción (*speedup* de 4,28 frente a 3,77). Con $n = 900$ y $p = 16$ gana `Gather` (2,19 frente a 1,74). La Figura 6 muestra los pasos 5 a 7 con $n = 1797$: en ambas versiones pasan de unos 2 ms con $p = 1$ a 13 ms con $p = 16$, y con $p = 8$ la reducción es más lenta (12 frente a 6 ms). La razón es que, con $p \leq 16$ y $k = 3$, el maestro revisa a lo más 48 candidatos por punto de prueba, así que la reducción ahorra poco. A cambio, cada nivel del árbol serializa objetos y ejecuta $\oplus$ en Python. Esto coincide con lo que anticipó el profesor: el ahorro es pequeño [8]. La reducción convendría con muchos más procesos o con un k mayor.

Otras mejoras que se desprenden de las mediciones:

- **Ordenamiento parcial en el paso 4.** El ordenamiento ocupa alrededor del 40 % del cómputo con $p = 1$ (Sección 3.3). Con `np.argpartition` se obtienen los k menores en tiempo lineal, sin ordenar todo el bloque. Esto reduce T_{comp_1}, aunque cambia el término $\log(n_{tr}/p)$ de la Tabla 3.

- **Colectivas con arreglos de NumPy.** Las funciones en mayúscula de mpi4py (`Bcast`, `Scatterv`, `Gather`) envían los arreglos sin serializarlos [3]. Esto bajaría la latencia medida, que hoy está dominada por la serialización, y la comunicación con $p = 1$. `Scatterv` permite bloques de distinto tamaño.
- **Reparto según la velocidad de cada núcleo.** Con núcleos de dos tipos, bloques iguales no equivalen a carga balanceada. Se puede fijar cada proceso a un núcleo, o dar bloques más grandes a los núcleos de rendimiento.
- **Ejecución en un clúster.** Medir en Khipu o en el laboratorio L507, con núcleos iguales y más de 16, permitiría verificar $p^{*\prime}$ y la isoeficiencia de la reducción.
- **Versión híbrida.** Combinar MPI entre nodos con hilos dentro de cada nodo. No la implementamos, porque el enunciado pide memoria distribuida.

Bibliografía

[1] J. Fiestas, *Computación Paralela y Distribuida: Proyecto – Laboratorio*, 2026-II. Lima, Perú: UTEC, 2026.
Impacto: define el caso 5 y el contenido del informe. De aquí tomamos el DAG (Figura 1) y la distribución de datos de la Introducción (*broadcast* de prueba, *scatter* de entrenamiento, *gather* de candidatos).

[2] `knn_digits_sec.py`, código secuencial del caso 5, adjunto al enunciado [1]. [Online]. Available: `https://drive.google.com/file/d/1wTBSh7bwT1QVvce0pFMv5tcCWsg4-HeK/view` (accessed Sep. 18, 2026).
Impacto: es el código que paralelizamos. De él obtuvimos T_s (Sección 2.1), los parámetros ($k = 3$, división 80/20, `random_state=42`) y la precisión de referencia (0,9833).

[3] L. Dalcin, *MPI for Python (mpi4py) documentation* [Online]. Available: `https://mpi4py.readthedocs.io/en/stable/` (accessed Nov. 4, 2026).
Impacto: revisamos cómo funcionan `comm.bcast`, `comm.scatter` y `comm.gather`, y que las reducciones de objetos usan un árbol por defecto (`mpi4py.rc.fast_reduce`), lo que sustenta el Algoritmo 2. También explica que las funciones en minúscula serializan los objetos, lo que usamos para interpretar la latencia medida y la comunicación con $p = 1$ (Secciones 3.1 y 3.3), y para proponer las colectivas con arreglos de NumPy (Sección 3.8).

[4] F. Pedregosa *et al.*, "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
Impacto: es la librería que usa el código base (`load_digits`, `train_test_split`). De ella salen los datos y su división, descritos en la Introducción.

[5] J. Fiestas, diapositivas y apuntes del curso CS4052: U1.1 Fundamentos de paralelismo, U1.2 Fundamentos (Foster, Flynn), U1.3 DAG, U2.1 Diseño en paralelo (*performance*), U2.2 PRAM 01, U2.3 PRAM 02, U2.4 Particionamiento, U3.2, U3.5, U6.1 y U6.2 (*performance*). Lima, Perú: UTEC, 2026.
Impacto: son la base teórica del informe: paralelización natural (Introducción), pasos de Foster y particiones balanceadas (Sección 2.2), formato del PRAM con **pardo**, pasos numerados y CREW (Sección 2.3), teorema de Brent (Sección 2.4), y $S = T_s/T_p$, $E = S/p$, isoeficiencia, FLOPs, medición de a y b y el modelo $T_{comm} = f(p)(\alpha + x\beta)$ (Sección 2.5). En los resultados seguimos su ping-pong para medir a y b (Sección 3.1), la granularidad G y el cruce de cómputo y comunicación (Secciones 3.3 y 3.5), y su método para validar la isoeficiencia escalando n (Sección 3.7).

[6] J. Fiestas, *CS4052 – Práctica Dirigida 02 (solución)*. Lima, Perú: UTEC, 2026.
Impacto: nos sirvió de modelo para escribir el pseudocódigo con Scatter, Broadcast y Gather (Sección 2.3) y para expresar la eficiencia en la forma $E = O(1/(1 + \dots))$ (Sección 2.5).

[7] R. Thakur, R. Rabenseifner y W. Gropp, "Optimization of Collective Communication Operations in MPICH," *International Journal of High Performance Computing Applications*, vol. 19, no. 1, pp. 49–66, 2005.
Impacto: describe el modelo de árbol binomial para las colectivas, que comentamos en la Sección 2.5 como explicación física de por qué el maestro recibe $(p-1)$ bloques en el *gather*. En la Sección 3.4 usamos su término $n(p-1)$ para ajustar la comunicación medida.

[8] J. Fiestas, respuesta por correo electrónico a consulta del grupo sobre el costo de `scatter` y `gather` y el acceso a Khipu, 6 de noviembre de 2026 (comunicación personal).
Impacto: confirma que el costo correcto es $O(\log p)$ y no $O(p)$, lo que sustenta la Sección 2.5, la Tabla 3 y la Figura 1. También autorizó medir en una laptop (Sección 3.1) y anticipó que la reducción ahorraría poco, lo que contrastamos con las mediciones (Sección 3.8).

[9] C. R. Harris *et al.*, "Array programming with NumPy," *Nature*, vol. 585, pp. 357–362, 2020.
Impacto: es la librería con la que se calculan las distancias en el código paralelo. Su manejo de arreglos temporales explicó el problema de rendimiento descrito en la Sección 3.2.

[10] Anthropic, *Claude Opus 5*, *Claude Sonnet 5* y *Claude Opus 5.5* (asistentes de inteligencia artificial, usados con Claude Code), setiembre y noviembre de 2026.
Uso: en la entrega parcial, resumió y ordenó el material de clase y redactó el borrador del informe. En esta entrega:

- Redactó en LaTeX la versión corregida del Método según las observaciones del profesor: pseudocódigo y diagramas (Secciones 2.2 y 2.3), trabajo y *span* (Sección 2.4) y métricas (Sección 2.5). Después aplicó la corrección de *scatter* y *gather* a $O(\log p)$ que confirmó el profesor [8].
- Escribió el código de las versiones beta y los scripts de validación, experimentos y gráficas. Detectó los problemas de medición de la Sección 3.2, incluido el de la ejecución que se colgó.
- Comparó el Método con las diapositivas del curso [5].
- Redactó la sección de Resultados a partir de las mediciones y de los coeficientes calculados por los scripts, y armó la presentación final.

Cada cifra de los Resultados se contrastó con los archivos de mediciones. Las mediciones las ejecutó el grupo en su laptop.
Impacto: la mayor parte de la redacción y del código se hizo con esta herramienta, lo que ahorró tiempo. También propuso la reducción en árbol para optimizar el paso 6 (Algoritmo 2). El grupo es responsable de lo presentado.